Chapitre 01 — Le shell et son environnement

Objectif Te repérer dans le terminal, lancer une commande et créer ton espace de travail. Distinguer les principaux shells et observer un sous-shell. Tu n'as encore aucun script à écrire.

1. Où taper les commandes ?

Depuis **PowerShell sur Windows**, connecte-toi à ta VM :

```
ssh stagiaire@192.168.84.10
```

La suite se déroule dans le terminal **Debian**, après la connexion. Adapte l'adresse si ton formateur t'en donne une autre. Le mot de passe ne s'affiche pas pendant la saisie.

Une ligne semblable à celle-ci attend ta commande :

```
stagiaire@debian:~$
```

Ne recopie pas cette invite ni le signe `$` : tape seulement la commande.



Du terminal à la commande exécutée

Le **terminal** est la fenêtre. Le **shell** est le programme qui interprète tes commandes. Ici, ce shell est **Bash**. SSH transporte tes échanges avec la VM.

2. Plusieurs shells, un même rôle

Linux peut utiliser plusieurs interpréteurs de commandes :

Nom	Repère pour débuter
`sh`	Nom historique et interface de référence pour les scripts portables. Sur notre Debian, `/bin/sh` lance **dash**.
`bash`	Shell utilisé dans cette formation. Il propose des fonctions supplémentaires, comme les tests `[[ ... ]]`.

Nom	Repère pour débuter
`zsh`, `fish`	Autres shells, avec leurs propres fonctions et réglages. Ils ne sont pas nécessaires pour ce parcours.

Vérifie les shells déclarés sur la VM, le programme utilisé pour `sh` et ton shell actuel :

```
cat /etc/shells
readlink -f /bin/sh
ps -p $$ -o comm=
```

- `/etc/shells` liste les shells de connexion déclarés ; la liste varie selon la machine.
- `readlink -f` suit le lien jusqu'au programme réel. Ici, le résultat est `/usr/bin/dash`.
- `ps` affiche un processus ; `-p $$` sélectionne le shell actuel, et `-o comm=` affiche son nom sans en-tête. Ici : `bash`.

La variable `$SHELL` indique normalement ton shell de connexion configuré. Elle ne permet pas de savoir avec certitude quel interpréteur exécute la commande actuelle : lancer `sh` depuis Bash ne la met pas automatiquement à jour.

Un shell est **interactif** quand il attend tes commandes, comme après notre connexion SSH. Il est **non interactif** quand il exécute un script ou une commande fournie avec `-c`. Un shell **de connexion** initialise aussi l'environnement de la session : c'est une autre distinction. Pour l'instant, retiens seulement que ces modes peuvent lire des fichiers de configuration différents ; `~/.bashrc` sert notamment aux réglages des sessions Bash interactives.

> `sh` et Bash ne sont pas interchangeables Nos futurs scripts utiliseront Bash. Les lancer avec `sh script.sh` peut échouer s'ils emploient des fonctions propres à Bash. Nous apprendrons à choisir l'interpréteur avec le **shebang**, au chapitre 03.

3. Un shell peut en lancer un autre

Après SSH, tu travailles dans un shell. Si tu tapes `bash`, il démarre un **nouveau Bash enfant**. La commande `exit` termine ce Bash et te ramène dans le parent. Un nouvel `exit` dans le parent ferme alors la connexion SSH.

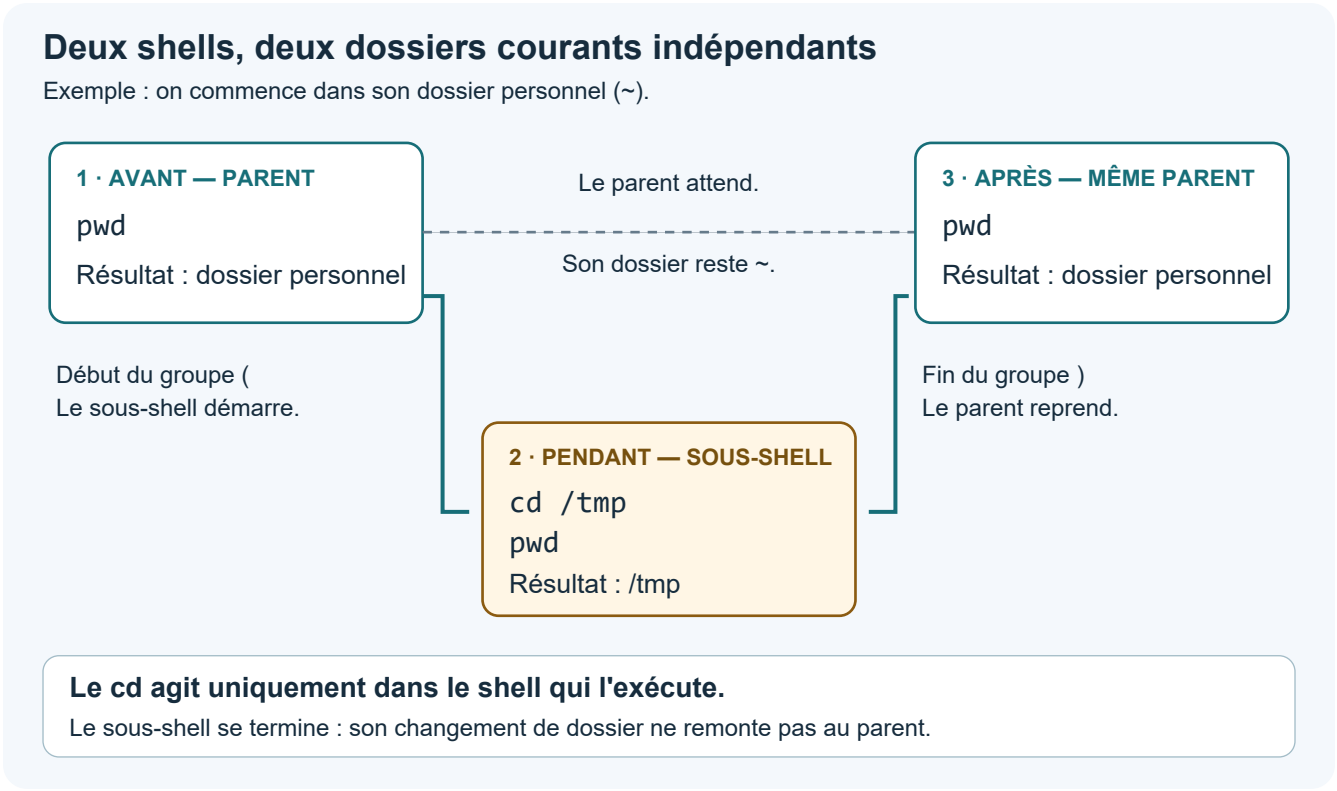
Pour une seule commande, `bash -c 'commande'` démarre un Bash enfant, exécute le texte fourni puis se termine :

```
bash -c 'echo "Bonjour depuis le Bash enfant"'
echo "De retour dans le shell parent"
```

Le shell enfant est un processus distinct. Il ne peut pas changer le dossier courant de son parent.

Les parenthèses permettent aussi d'exécuter un groupe de commandes dans un **sous-shell**, avec une copie de l'environnement du shell courant :

Imagine une cabine qui reçoit une copie de ton poste de travail. Elle peut changer son propre dossier sans déplacer ton poste. **Le parent attend pendant que la cabine travaille, puis reprend quand elle se ferme.** On reste sur la même VM : aucune nouvelle connexion SSH n'est ouverte.



Avant, pendant et après : le sous-shell change son dossier vers /tmp, le parent conserve son dossier personnel

Prédis avant de lancer : le dernier `pwd` affichera-t-il `/tmp` ou ton dossier personnel ? Dans cet exemple, `cd ~` place d'abord le parent dans ton dossier personnel.

```
cd ~
pwd
(
    cd /tmp
    pwd
)
pwd
```

Vérifie les trois résultats :

```
/home/stagiaire
/tmp
/home/stagiaire
```

Sur notre VM, le dossier personnel est `/home/stagiaire` ; adapte ce nom si tu utilises un autre compte. Le premier et le dernier résultat sont identiques. Le `cd /tmp` a agi seulement dans le sous-shell.

À ton tour : remplace `/tmp` par `/etc` dans le groupe. Prédis les trois lignes, puis teste. Seule la ligne du milieu doit changer.

Les variables suivront une logique similaire : nous verrons leur transmission avec `export` au chapitre 02.

4. Une commande, ses options et ses arguments

```
ls -l /tmp
```

Élément	Ce qu'il demande
`ls`	Lister un dossier
`-l`	Afficher les détails : c'est une option
`/tmp`	Choisir le dossier à lister : c'est un argument

Une commande se termine quand tu appuies sur **Entrée**. Bash distingue les majuscules et les minuscules : `ls` et `LS` sont deux noms différents.

5. Se repérer avant d'agir

```
whoami
hostname
pwd
```

- `whoami` : ton compte ; ici, normalement `stagiaire`.
- `hostname` : le nom de la machine distante.
- `pwd` : le dossier où tu travailles ; au départ, normalement `/home/stagiaire`.

Crée un espace réservé à cette formation :

```
mkdir -p ~/atelier-bash/decouverte
cd ~/atelier-bash/decouverte
pwd
```

`~` désigne ton dossier personnel. `-p` crée aussi les parents manquants et accepte qu'un dossier existe déjà. `cd` change le dossier courant.

Chemin	Signification
`/home/stagiaire`	Chemin absolu, depuis la racine `/`
`decouverte`	Chemin relatif au dossier courant
`.`	Dossier courant
`..`	Dossier parent
`~`	Dossier personnel

```
cd ..
pwd
cd decouverte
```

Tu reviens maintenant dans `~/atelier-bash/decouverte`.

6. Écrire puis lire un fichier

```
echo "Station Orion ouverte" > journal.txt
cat journal.txt
```

Résultat :

```
Station Orion ouverte
```

`echo` affiche le texte puis ajoute automatiquement un retour à la ligne. Le signe `>` dirige ce texte vers un fichier : **il crée le fichier ou remplace son contenu**.

Pour ajouter une ligne :

```
echo "Radio disponible" >> journal.txt
cat journal.txt
```

`>>` ajoute à la fin. Il ne remplace pas les lignes déjà présentes.

> Avant d'utiliser `>` Vérifie le nom du fichier et ton dossier avec `pwd`. Dans le cours, nous écrivons seulement dans notre espace de travail.

7. Passer un résultat à une autre commande

```
cat journal.txt | grep Radio
```

Le tube `|` transmet la sortie de `cat` à `grep`. `grep` garde les lignes contenant `Radio`. Le fichier d'origine reste intact.

On peut aussi demander directement à `grep` de lire le fichier :

```
grep Radio journal.txt
```

Ces deux exemples affichent `Radio disponible`. Le premier montre comment relier deux commandes ; le second est plus court pour ce besoin précis.

8. Les raccourcis utiles

Geste	Effet
Tab	Complète un nom ; deux appuis peuvent proposer les choix
Flèche haut	Rappelle une commande précédente
Ctrl+C	Interrompt la commande en cours
Ctrl+L	Nettoie l'affichage sans effacer les fichiers
`exit`	Ferme la session SSH

L'aide de `ls` se consulte avec `ls --help`. Pour une commande intégrée à Bash, comme `cd`, utilise `help cd`. Le lecteur `man ls` se quitte avec **q**.

9. Pour aller plus loin : un alias temporaire

Un alias donne un raccourci à une commande. Cet exemple agit seulement dans ton shell actuel :

```
alias ll='ls -l'
ll
unalias ll
```

Nous ne modifions pas encore les fichiers de configuration du shell. Plus tard, `~/.bashrc` permettra de conserver des réglages pour les shells Bash interactifs qui le lisent.

À retenir

- Tu tapes dans un terminal ; Bash interprète les commandes sur Debian.
- Sur cette VM, `sh` lance dash ; nos scripts seront écrits pour Bash.
- Le `cd` effectué dans un sous-shell ne change pas le dossier du parent.
- `pwd` permet de vérifier où tu es avant d'écrire un fichier.
- `>` remplace ; `>>` ajoute ; `|` transmet un résultat.

À toi : quelle différence entre `cd ..` et `cd ~` ? Que contient le journal après deux ajouts ? Pourquoi le `cd /tmp` entre parenthèses ne te déplace-t-il pas définitivement ?

Passer au TP